

Stat 437 HW4

John Salmon (011745357)

General rule

Please show your work and submit your computer codes in order to get points. Providing correct answers without supporting details does not receive full credits. This HW covers

- Bayes classifier
- kNN classifier

For an assignment or project, you DO NOT have to submit your answers or reports using typesetting software. However, your answers must be well organized and well legible for grading. Please upload your answers in a document to the course space. Specifically, if you are not able to knit a .Rmd/.rmd file into an output file such as a .pdf, .doc, .docx or .html file that contains your codes, outputs from your codes, your interpretations on the outputs, and your answers in text (possibly with math expressions), please organize your codes, their outputs and your answers in a document in the format given below:

Problem or task or question ...

Codes ...

Outputs ...

Your interpretations ...

It is absolutely not OK to just submit your codes only. This will result in a considerable loss of points on your assignments or projects.

Conceptual exercises: I (Bayes classifier)

1. This exercise is on Bayes theorem and Bayes classifier.

1.1) State clearly the definition of the 0-1 loss function. Can this function be used in multi-class classification problems?

The 0-1 loss function is a measure used to evaluate the accuracy of a two class classification task. It operates as a function that returns one of two possible values, 0 or 1. Given a true class label $G(x)$ and a predicted class value $\hat{G}(x)$ it returns 0 if $(G(x) = \hat{G}(x))$, meaning the predicted label matches the true label (correctly classified) and 1 otherwise meaning the predicted label does not match the true label (incorrectly classified).

0-1 loss can be used in multi-class classification problems because very simply, it just counts the number of classifications errors made. However it is not the best method to use because it is difficult to optimize.

1.2) Let Y be the random variable for the class label of a random vector X , such that $Y \in \mathcal{G} = \{1, \dots, K\}$ where $K \geq 2$ is the number of classes. Let $\hat{Y}$ be the estimated class label for X . Given the prior $\Pr(Y = k) = \pi_k, k \in \mathcal{G}$ on Class k and the conditional density $f_k(x)$ of X when it comes from Class k . Provide the formula to obtain the posterior $\Pr(Y = k|X = x)$, which is essentially the Bayes theorem. What is the Bayes classifier and how does it classify a new observation x_0 from X ? Is the decision boundary of the Bayes classifier linear or quadratic in X ? Explain (but do not have to mathematically prove) why the Bayes classifier minimizes the expected 0-1 loss. Note the a proof of the fact that the Bayes classifier minimizes the expected 0-1 loss is given in “LectureNotes4_notes.pdf”. You should not copy and paste the proof. Instead, please provide the explanation based on your understanding of the proof.

Posterior Probability Formula:

$$\Pr(Y = K|X = x) = \frac{\pi_K f_K(x)}{\sum_{j=1}^K \pi_j f_j(x)}$$

The Bayes Classifier is the classification algorithm that assigns observations to a class based on their calculated posterior probability of belonging to that class. The class with the highest probability of belonging to is the one that is chosen by the classifier. The shape of the decision boundary is based on the class densities and their covariance properties. The decision boundary is linear if the class densities are Gaussian and have equal covariance matrices. The Bayes Classifier minimizes the expected 0-1 loss because of the nature of the algorithm and 0-1 loss function. For any given x there are possible class labels each with a different probability of being assigned. The best strategy for minimizing the chance of misclassification is to pick the class with the highest likelihood of being correct. This is essentially what the Bayes classifier does and is also why it is considered the theoretical best classification method.

1.3) If $K = 2$ in subquestion 1.2), what is the threshold value on $\Pr(Y = 1|X = x_0)$ that is used by the Bayes classifier to determine the class label for x_0 ? Suppose you use a different threshold value on $\Pr(Y = 1|X = x_0)$ to classify x_0 , is the corresponding classifier still the Bayes classifier, and is the corresponding loss function still the 0-1 loss? Explain your answer. Provide a scenario where to classify an observation a different threshold value is more sensible than the threshold value used by the Bayes classifier.

The threshold value is the value that determines how the classifier makes a decision. In the case where $K = 2$ the threshold should be set at $\Pr(Y = 1|X = x_0) = 0.5$ meaning that the classifier assigns observations based on which ever class has a probability greater than 0.5 or 50%. In the case where it is an exact 50/50 split, its often just assigned to class 1 or it can simply be not classified at all depending on how the user sets up the algorithm.

If you choose a different threshold value than 50/50 it is no longer considered the Bayes Classifier because it is not choosing the class based on which one has the highest probability. The corresponding loss function would also not be considered the 0-1 loss function and instead be replaced with a different loss that might be more sensitive to one type of error such as a false positive or false negative, which take on more meaning in this new case. Once scenario where utilizing a different threshold value may be useful is in cases where false positives and false negatives have different consequences, or the stakes of being incorrect or correct are higher. Such as in airport security where missing an explosive device smuggled in a passengers luggage could cause a mass casualty

event while falsely identifying a dangerous device in a passengers luggage leads to a search. In this case you may adjust the threshold to trigger a search in any case of doubt to make sure no explosive devices are brought onto passenger airplanes.

1.4) If $K = 2$ in subquestion 1.2), $\pi_1 = 0.6$, $f_1(x) \sim \text{Gaussian}(0, 1)$ and $f_2(x) \sim \text{Gaussian}(2, 1)$ and $x_0 = 1.5$. Compute $\Pr(Y = 1|X = x_0)$ and use the Bayes classifier to classify x_0 .

$$f_1(1.5) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1.5^2}{2}} = 0.12951 \dots$$

$$f_2(1.5) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(1.5-2)^2}{2}} = 0.3520 \dots$$

$$\Pr(Y = 1|X = 1.5) = \frac{0.6(0.12951)}{(0.6(0.12951))(1-0.6(0.3520))} = 0.356$$

$0.356 \leq 0.5$ The probability of the observation belonging to class 1 is lower than the 0.5 threshold so the Bayes classifier would classify the observation into class 2.

Conceptual exercises: II (k -NN classifier)

2. Given the training set $\mathcal{T}$ of n observations $(x_1, y_1), \dots, (x_n, y_n)$, where y_i is the class label of observation x_i and $y_i \in \mathcal{G} = \{1, \dots, K\}$ for $K \geq 2$, consider k -NN classifier, where k is the neighborhood size.

2.1) Describe how the decision boundary (such as its smoothness and shape) of k -NN classifier changes as k changes.

As the number of nearest neighbors denoted by k changes, the decision boundary will change in resolution and smoothness. As the value of k increases, the decision boundary will become smoother, for example having fewer jagged edges or points. As k the boundary will become more volatile and rough. Except for at $k = 0$ and $k = n$. $k = 0$ simply just isn't possible because the solution set would be empty, at $k = n$ there would not be a decision boundary because each point would be assigned to the class that has the most observations.

2.2) Explain why the training error of 1-NN classifier is 0. Provide an estimator of the test error of a classifier and explain why it can be used as such an estimator. Is it true that a large k leads to a k -NN classifier with smaller test error? Can the test error of a k -NN classifier be equal to the test error of the Bayes classifier? When k is large and k/n is small, what is a k -NN classifier approximately estimating?

When $k = 1$ the points nearest neighbor will always be itself. While training the error will be zero because the classification is based on the known class of each single point, and each point will always be assigned correctly. In order to provide an estimate of the test error of a classifier, we have to use a training and testing set often employing cross validation. This involves running multiple tasks on different folds of the data set to obtain an error rate, these error rates are then averaged.

In general, increasing the value of k does not guarantee that the test error of the model will decrease. A larger value of K will reduce the variance but also increases the bias. The optimal k value is one where the classifier has a good balance between its variance and its bias.

The Bayes classifier is the most optimal classification algorithm. The test error of the kNN classifier can approach the Bayes error as the sample size grows towards infinity and the value of k is appropriately assigned, however in the real world where we don't have infinite observations the kNN classifier will have a higher test error.

When k is large but $\frac{k}{n}$ is small, the classifier is approximating the Bayes Posterior Probability $P(Y|X)$, this means that it is estimating the probability distribution of class labels rather than making decisions based on the neighbors values.

2.3) When there are $K \geq 2$ classes, how does a k -NN classifier classify a test observation x_0 ?

When there are greater than two classes the process is very much the same. The algorithm computes the distances, finds the k nearest neighbors based on the distance, and then uses a majority vote to classify the observation. If there is a tie, the observation can just be assigned to the closest neighbors class, or just not classified.

2.4) When should data be standardized before applying a k -NN classifier? When standardizing data, do we standardize each observation or each feature?

Because kNN uses distance based metrics, the features need to be relatively similar in scale. This means you should standardize your data when the scales have vastly different ranges, or you have one feature that is incredibly large compared to the others. When standardizing you should do it column-wise so that each feature for each observation ends up on the same scale.

2.5) Using your understanding of Example 3 in “LectureNotes4b_notes.pdf”, provide a step-by-step guide on how to choose an optimal k for k -NN classifier using cross-validation. You can provide such as guide in the form of “pseudocode” (see, e.g., <https://en.wikipedia.org/wiki/Pseudocode> for some details on pseudocode). Suppose the training set has few observations, can you still perform cross-validation in order to choose an optimal k ? Explain your answer. (Hint: for the 2nd part, think about if having more observations helps better estimate test error.)

Prepare Dataset split dataset into 5 - 10 folds (depends on user choice) with training and validation sets.

declare a range of k values to try (1-100?, 1-1,000?) for each k value: for each fold: perform Knn compute the test error estimate average the test error from each fold store the average test error associated with the value of K Find and return the best test error to find which value of K performed the best

If the training set doesn't have many observations, you can still use it to try and find the optimal K value. However, the value it chooses may not be the correct one. A smaller test set means there will likely be more variance in the estimated test error metrics. Meaning the value of k may not work well once its applied to new data.

Applied exercises: I (k -NN classifier)

7. Please refer to the NYC flight data `nycflights13` that has been discussed in the lecture notes and whose manual can be found at <https://cran.r-project.org/web/packages/nycflights13/index.html>. We will use `flights`, a tibble from `nycflights13`.

Please use `set.seed(123)` for the whole of this exercise. Randomly select from `flights` for each of the 3 carrier “UA”, “AA” or “DL” 500 observations for the 3 features `dep_delay`, `arr_delay` and `distance`. Let us try to see if we can use the 3 features to identify if an observation belongs a specific carrier. The following tasks and questions are based on the extracted observations. Note that you need to remove rows with `na`'s from the extracted observations.

```
set.seed(123)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(nycflights13)
flights = na.omit(flights)
flights = flights %>% filter(carrier %in% c('UA', 'AA', 'DL'))
flights = flights %>% select(dep_delay, arr_delay, distance, carrier)
sampled_flights = flights %>% group_by(carrier) %>% sample_n(500) %>% ungroup()
```

7.1) First, you need to standardize the features since they are on very different scales. Then randomly split the observations into a training set that contains 70% of the observations and a test set that contains the remaining observations.

```
y = sampled_flights$carrier
x = sampled_flights[,1:3]

x = scale(x)

train_index <- sample(1:nrow(x), size = 0.7 * nrow(x))
x_train <- as.data.frame(x[train_index, ])
y_train <- as.data.frame(y[train_index])

x_test <- as.data.frame(x[-train_index, ])
y_test <- as.data.frame(y[-train_index])
```

7.2) Consider the observations as forming 3 classes that are determined by `carrier`. To the training set, apply 10 fold cross-validation to k -NN classifier with features `arr_delay`, `dep_delay`, and `distance` to determine the optimal k from the values $\{1, \dots, 15\}$. Apply the optimal k -NN to the test set, provide the classification table and the overall error rate, and provide visualization of the classification results. Do you think the error rate is reasonable? Explain your answer. (Hint: you can follow the strategy provided by Example 3 in “LectureNotes4b_notes.pdf”.)

```
library(class)
library(ggplot2)
m = 10 #10 folds
folds = sample(1:m, nrow(x_train)/m, replace = TRUE)
table(folds) #check fold balance
```

```
## folds
##  1  2  3  4  5  6  7  8  9 10
## 15 14 10 14  9  3  7  9 11 13
```

```
kmax = 5
testErrors = matrix(0,nrow=2,ncol=kmax)
for (k in 1:kmax){
  testError1 = double(m)
  for (s in 1:m){
    trainingTmp = x_train[folds !=s,]
    testTmp = x_test[folds==s,]
    trainingLabsTmp = y_train[folds !=s,]
    testLabsTmp = y_test[folds==s,]
    knn = knn(trainingTmp, testTmp, trainingLabsTmp, k)
    n_mistakes = sum(1-as.numeric(knn==testLabsTmp))
    test_error = n_mistakes/length(testLabsTmp)
    testError1[s] = test_error
  }
  testErrors[,k] = c(mean(testError1), sd(testError1))
}

colnames(testErrors) = paste("k=", 1:kmax,sep="")
rownames(testErrors)=c("mean(TestError)", "sd(TestError)")
print(testErrors)
```

```
##               k=1          k=2          k=3          k=4          k=5
## mean(TestError) 0.50194405 0.54207368 0.53165438 0.53049332 0.52599671
## sd(TestError)   0.07399225 0.05253948 0.07767966 0.07486965 0.07022573
```

```
khat = which(testErrors[1,] == min(testErrors[1,]))
cl = y_train[,1]
best_knn = knn(x_train, x_test, cl, khat)
n_errors = sum(1 - as.numeric(as.data.frame(best_knn)==y_test))
best_test_error = n_errors/length(y_test)
print(best_test_error)
```

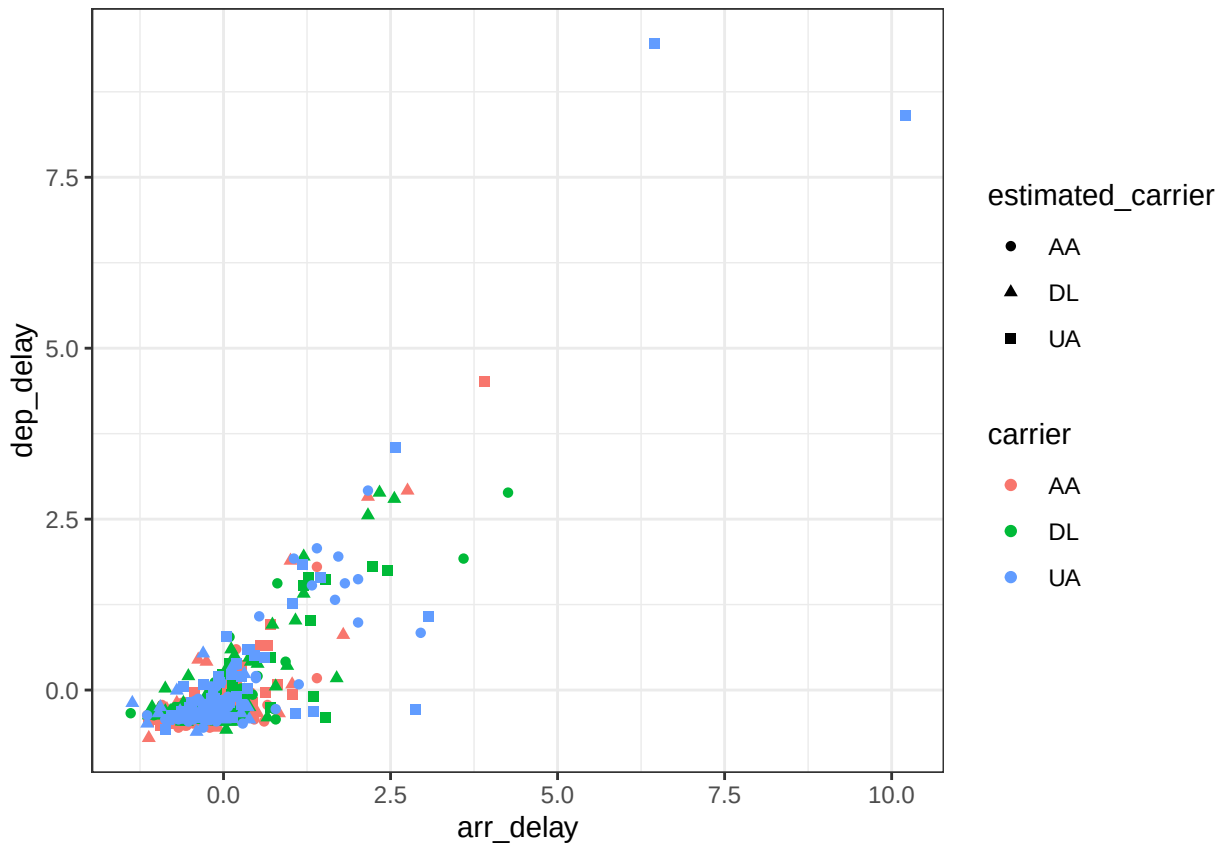
```
## [1] 227
```

```

x_test$carrier = factor(y_test[,1])
x_test$estimated_carrier = best_knn

p = ggplot(x_test, aes(arr_delay, dep_delay)) +
  geom_point(aes(shape = estimated_carrier, color = carrier)) + theme_bw()
p

```



7.3) Note that you have standardized the features **arr_delay**, **dep_delay**, and **distance**. However, with the unstandardized features, you would surely know that none of them follows a Gaussian distribution no matter with respect to which class (i.e., **carrier**) you look at their observations since these features have non-negative values (whereas a Gaussian distribution can generate negative values). Again, the 3 classes are determined by **carrier**. So, you will apply QDA based on the 3 standardized features to the training set to train the model, and then apply the trained model to the test set (that contains the 3 standardized features) to classify its observations.

(7.3a) First, check if the Gaussian assumption is satisfied. For this, note that if the standardized features **arr_delay**, **dep_delay**, and **distance** follow a trivariate Gaussian distribution for each individual class, then any pair among the 3 standardized features follows a bivariate Gaussian distribution for each individual class.

(7.3b) Apply the estimated (i.e., trained) QDA model to the test set, provide the estimated mixing proportion and estimated mean vector for each class, and provide the classification table. If you randomly pick an observation on the 3 standardized features, is it approximately equally likely to belong to each of the 3 carriers? (You do not need to mathematically prove your answer. However,

think along this line: we are testing the equality of 3 population proportions, and each estimated population proportion is based on around 350 observations, which approximately can be done via a z-test since the central limit theorem is in effect.) How is the performance of QDA on this test set? Explain your answers.

(7.3c) Extract observations that are for “UA” or “DL” from the training set and the test set, respectively, to form a new training set and a new subset, so that there are now 2 classes “UA” and “DL”. Apply QDA to the new training set and then apply the trained model to the new test set. Report the overall error rate on the test set, provide the ROC curve, and calculate the AUC. How is the performance of QDA on this test set? Explain your answer.